

République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
École Supérieure Privée d'Ingénierie et de Technologie — TEK-UP

Conception et développement d'une plateforme d'inventaire logistique multi-canal

Rapport de Projet de Fin d'Études — Lot 3

Chapitre 4 — Réalisation technique

Réalisé par : **Harrabi Yosri**

Spécialité : Génie Logiciel et Systèmes d'Information

Encadrant professionnel : M. Ferchichi Marwen — VECTORYS Tunisie

Encadrante académique : Mme Zrelli Siwar — TEK-UP

Année universitaire 2025 — 2026

Table des matières — Lot 3

Chapitre 4 — Réalisation technique

4.1 Environnement de développement et outillage

4.2 Réalisation de l'API ASP.NET Core

4.2.1 Organisation Clean Architecture et CQRS

4.2.2 Sécurité JWT et politiques d'autorisation

4.2.3 Persistance SQL Server et migrations

4.2.4 Endpoints métier et hub SignalR

4.3 Réalisation de l'application web de supervision

4.4 Réalisation de l'application mobile agent

4.4.1 Authentification et navigation

4.4.2 Boucle de scan et quotas

4.4.3 Mode hors ligne et synchronisation

4.5 Déroulement des sprints et livrables

4.6 Déploiement sur serveur local

4.7 Difficultés rencontrées et solutions retenues

Conclusion du chapitre

Chapitre 4 — Réalisation technique

Ce chapitre décrit la mise en œuvre concrète de la conception présentée au chapitre 3. Il détaille l'environnement de travail, l'implémentation de l'API centrale, des clients web et mobile, le déroulement des sprints Scrum, puis les choix de déploiement sur un serveur local adapté au contexte d'exploitation VECTORYS. Les illustrations reprennent les captures déjà intégrées aux chapitres précédents et réservent des emplacements pour les vues de déploiement.

Introduction

La réalisation a suivi une logique de *tranches verticales* : chaque sprint livre un bout de fonctionnalité traversant l'API, la base de données et au moins un canal client (web ou mobile). Cette approche réduit le risque d'un backend « orphelin » ou d'interfaces déconnectées du métier. Le fil conducteur demeure le cycle inventaire C1 → comptage → écarts → pilotage KPI.

Les sections suivantes s'organisent d'abord par couche technique (outillage, API, web, mobile), puis par chronologie des sprints, afin de relier les décisions de conception aux artefacts livrés dans le dépôt et aux interfaces réellement utilisées en démonstration.

4.1 Environnement de développement et outillage

Le poste de développement et le serveur de démonstration s'appuient sur un stack Microsoft/Flutter homogène avec les choix du chapitre 1.

Composant	Technologie / version	Rôle dans la réalisation
IDE / outils	Visual Studio / VS Code, Flutter SDK, SSMS	Édition, debug, inspection SQL
Backend	ASP.NET Core 9, C#, MediatR, FluentValidation	API REST + hub SignalR
ORM	Entity Framework Core + migrations	Schéma VectorysDB, seed utilisateurs
SGBD	Microsoft SQL Server Express (`YOSRI\SQLEXPRESS`)	Persistance centrale
Web supervision	Angular 19, TypeScript, Chart.js	Portail superviseur / KPI
Mobile agent	Flutter (Dart), Dio, SQLite	Scan terrain et sync offline
Excel	ClosedXML	Import du stock théorique

Docs API	Swagger / OpenAPI (profil Development)	Exploration et tests manuels
Qualité / logs	Serilog, middleware d'exceptions	Diagnostic runtime
Versionnement	Git	Historique des sprints et branches

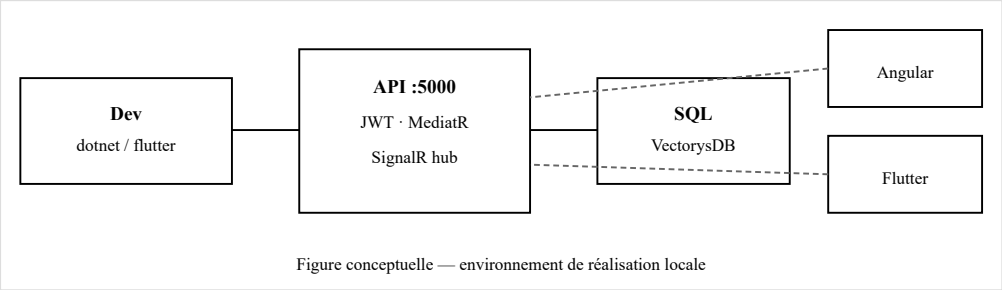


Figure 4.1 — Chaîne de développement locale (clients → API → SQL Server)

Un script PowerShell (`scripts/setup-database.ps1`) applique les migrations EF Core et prépare le seed des comptes de démonstration. Un fichier Excel de test (`scripts/test-import-stock.xlsx`) et un générateur dédié facilitent la reproductibilité des scénarios d'import et de scan.

4.2 Réalisation de l'API ASP.NET Core

L'API constitue le point unique de vérité métier. Tous les canaux consomment les mêmes commandes et requêtes ; aucune règle de quotas, d'écarts ou d'assignation n'est dupliquée côté client.

4.2.1 Organisation Clean Architecture et CQRS

La solution Visual Studio est découpée en quatre projets :

- **VectorysInventaire.Domain** — entités (*Inventaire*, *Passage*, *Emplacement*, *StockTheorique*, *Scan*, *Ecart*, *Utilisateur*) et enumérations de statut ;
- **VectorysInventaire.Application** — Features organisées en Commands/Queries (MediatR), DTOs, validateurs FluentValidation et contrats d'interfaces ;
- **VectorysInventaire.Infrastructure** — EF Core, repositories, JWT, ClosedXML, hash BCrypt, seeders ;
- **VectorysInventaire.API** — contrôleurs HTTP, hub SignalR, middleware, composition DI (`Program.cs`).

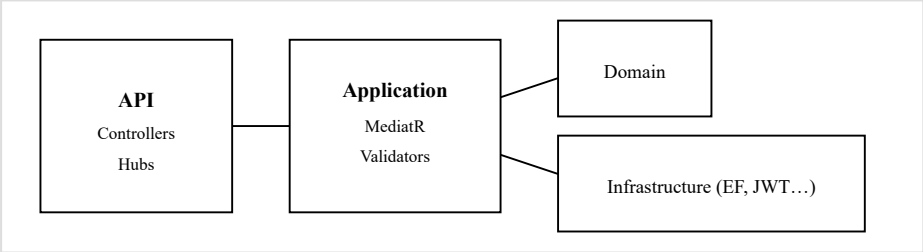


Figure 4.2 — Couches de la solution .NET réalisées

Exemple de flux typique : le contrôleur reçoit un DTO, dispatche une commande MediatR, le handler orchestre les repositories, puis un service de notification publie éventuellement un événement SignalR. Cette discipline évite le « fat controller » et facilite l'ajout de validations sans modifier le protocole HTTP.

4.2.2 Sécurité JWT et politiques d'autorisation

L'authentification est centralisée dans `POST /api/auth/login` :

- superviseur / administrateur : email + mot de passe ;
- agent : code personnel (identifiant terrain), avec vérification BCrypt du hash stocké.

Le service JWT émet un jeton de 8 heures pour les agents et de 24 heures pour les superviseurs. Les politiques `SuperviseurOnly` et `AgentOnly` protègent les contrôleurs. Le

hub SignalR lit le jeton via le query string `access_token` afin d'autoriser les connexions WebSocket depuis Angular.

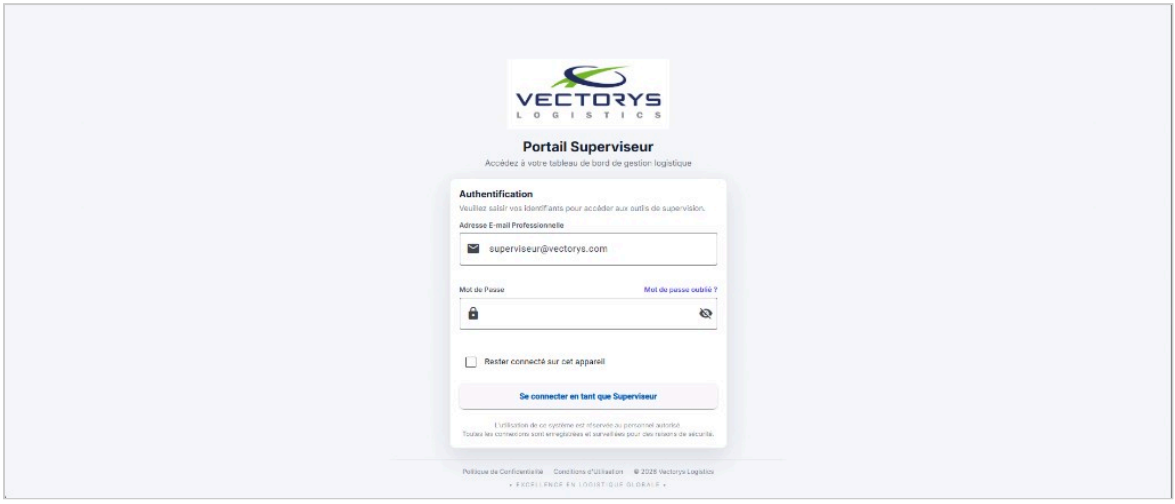


Figure 4.3 — Portail web : authentification superviseur (réalisation Angular)

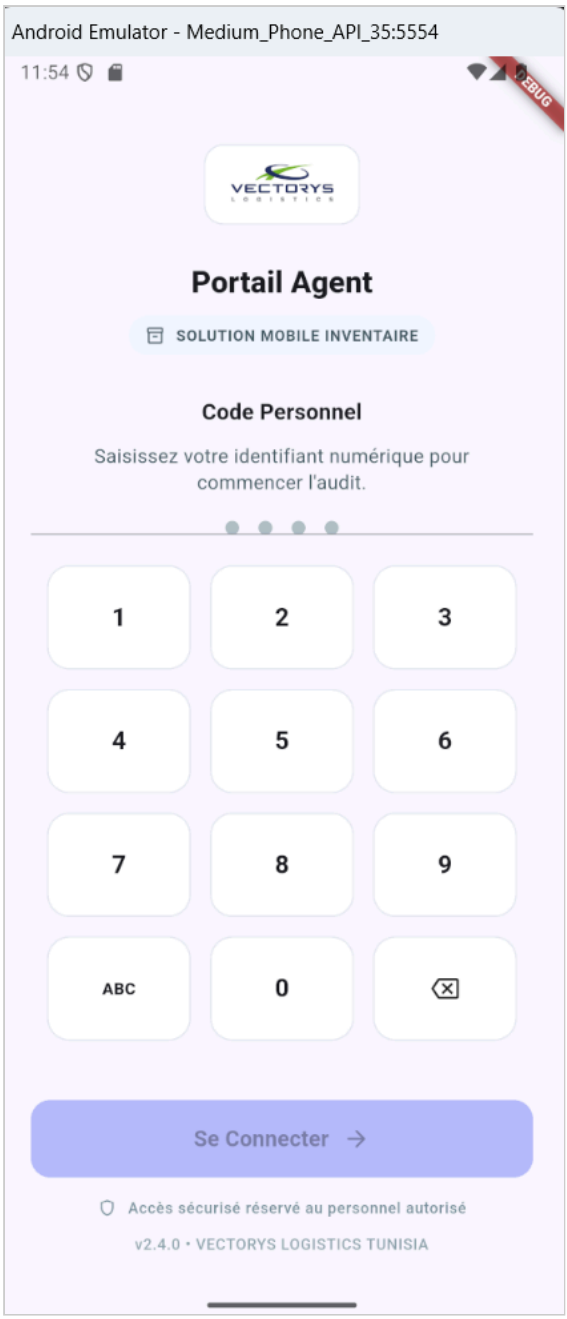


Figure 4.4 — Application mobile : authentification agent (réalisation Flutter)

4.2.3 Persistance SQL Server et migrations

Les migrations EF Core matérialisent l'évolution du schéma au fil des sprints :

Migration	**Contenu principal**
InitialCreate	Utilisateurs, fondations sécurité
Sprint2_InventaireSchema	Inventaires, passages, emplacements, stock théorique
Sprint3_ScansSchema	Scans, clés et relations comptage
Sprint5_EcartsSchema	Écarts MANQUANT / SURPLUS et statuts de traitement

Sprint7_KpiIndexes	Index en soutien aux agrégats KPI

Au démarrage, `Program.cs` applique automatiquement `MigrateAsync` puis exécute le seeder (comptes `superviseur@vectorys.local`, agents `AG001` / `AG002`, etc.). Cette stratégie simplifie la mise en route d'un environnement local neuf.

4.2.4 Endpoints métier et hub SignalR

Le tableau suivant synthétise les endpoints effectivement exposés par l'API réalisée.

Contrôleur	**Méthodes clés**	**Rôle métier**
Auth	POST login	Émission JWT multi-rôle
Inventaires	GET, POST, suivi-temps-reel	Création, liste, monitoring
Passages	POST, import-stock	C2 manuel ; import Excel ClosedXML
Emplacements	GET, mes-emplacements	Supervision / tâches agent
Scans	POST, batch, fin-comptage	Comptage online et sync offline
Ecarts	detecter, branche-a, GET	Détection et recomptage ciblé
Kpi	dashboard, precision, performance, qualite, clients, wms	Pilotage analytique
Agents	GET	Liste agents actifs pour assignation
Hub	/hubs/inventaire	Groupes par inventaire, push temps réel

Règle métier structurante (scan). Lors de `EnregistrerScan`, l'emplacement passe de *EN_ATTENTE* à *EN_COURS*, puis à *TERMINE* dès que l'ensemble des codes US a atteint la quantité théorique. La clôture manuelle terrain est volontairement désactivée côté mobile afin d'éviter les fins de comptage prématurées.

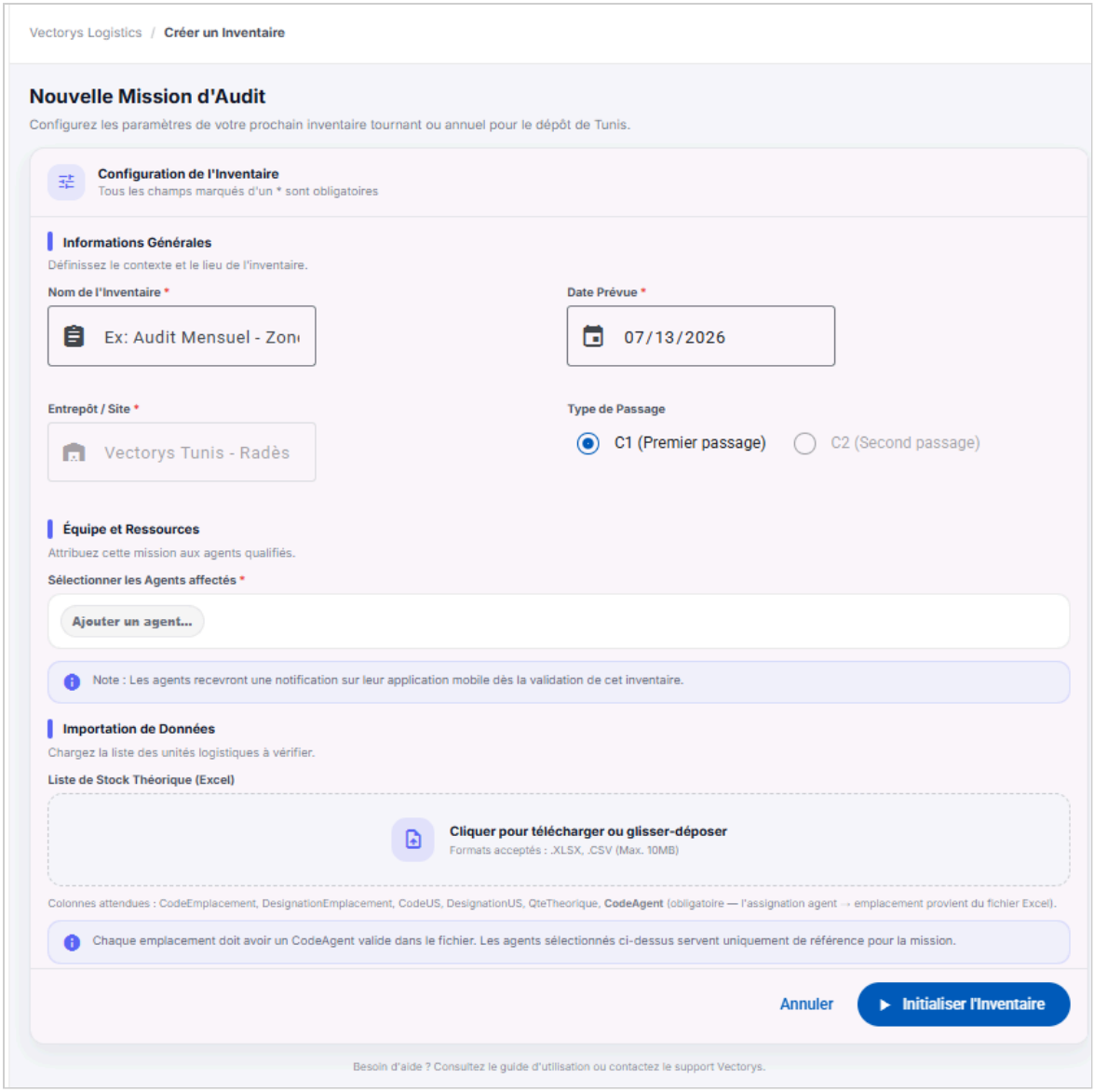


Figure 4.5 — Création d'inventaire et préparation de l'import (interface réalisée)

4.3 Réalisation de l'application web de supervision

Le portail Angular 19 s'adresse au superviseur (et à l'administrateur). Il consomme l'API via des services HTTP, stocke le JWT, protège les routes par garde d'authentification et affiche le suivi temps réel en s'abonnant au hub SignalR après sélection d'un inventaire.

Les modules fonctionnels réalisés côté supervision sont :

- **Authentification** — formulaire email / mot de passe, intercepteur Bearer, redirection vers le tableau de bord ;
- **Inventaires** — création (intitulé, zone, dates), liste paginée, statut EN_PREPARATION puis EN_COURS après import ;
- **Import Excel** — téléversement multipart vers `POST /api/passages/{id}/import-stock`, affichage du rapport (emplacements créés, assignations par CodeAgent) ;
- **Monitoring** — avancement des agents et des emplacements, notifications de nouveaux scans ;
- **Écarts** — détection C1, consultation MANQUANT / SURPLUS, déclenchement de la branche A (recomptage ciblé C2) ;
- **KPI** — dashboard agrégé (précision, performance, qualité, clients, WMS) avec filtres et graphiques Chart.js.

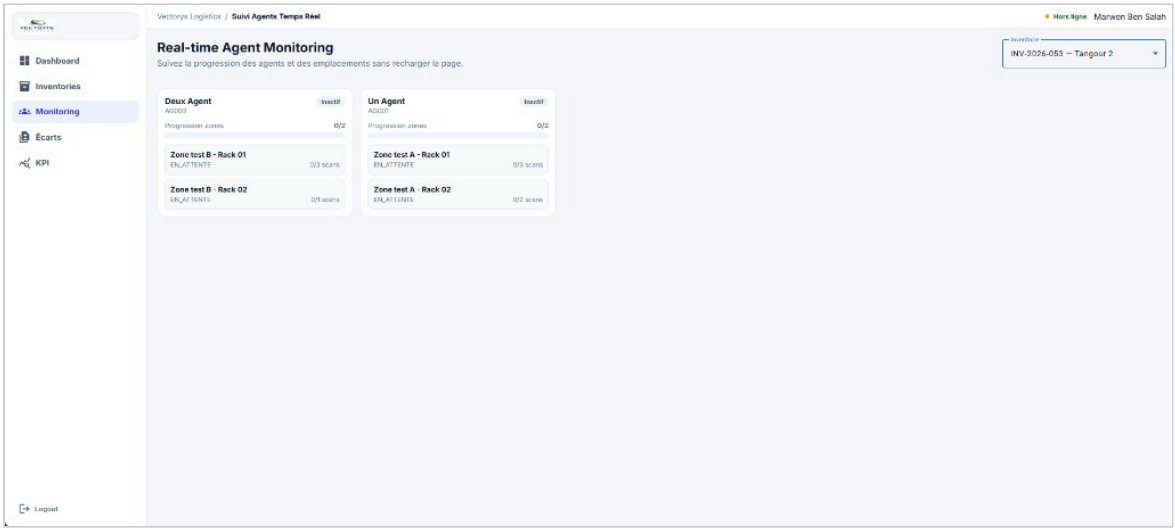


Figure 4.6 — Suivi d'avancement temps réel (canal web)

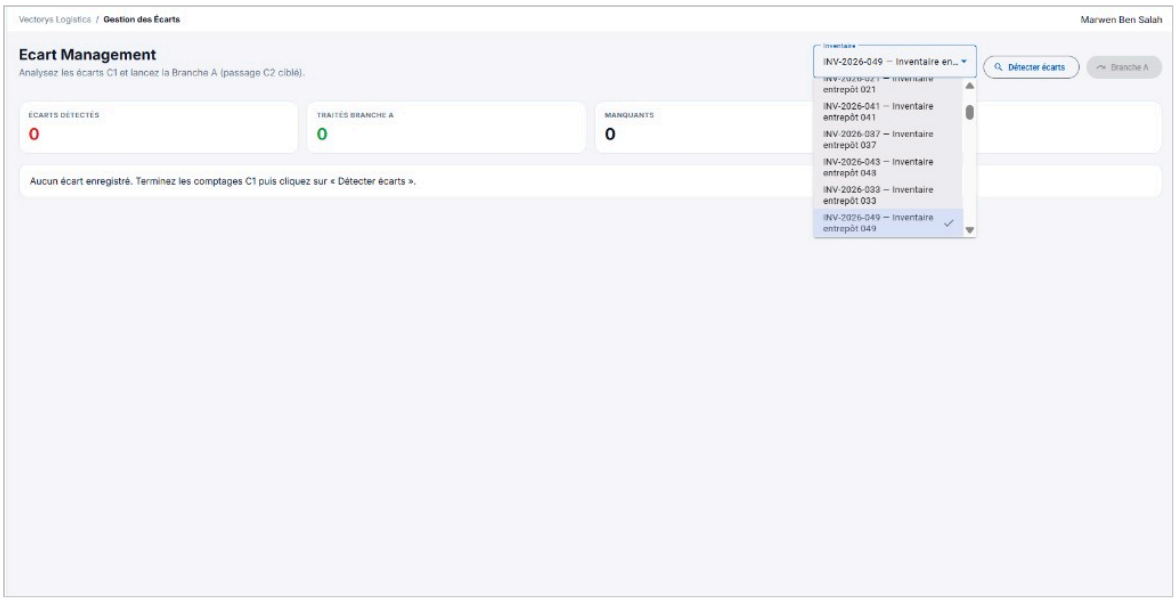


Figure 4.7 — Gestion des écarts et stratégie de recomptage (branche A)

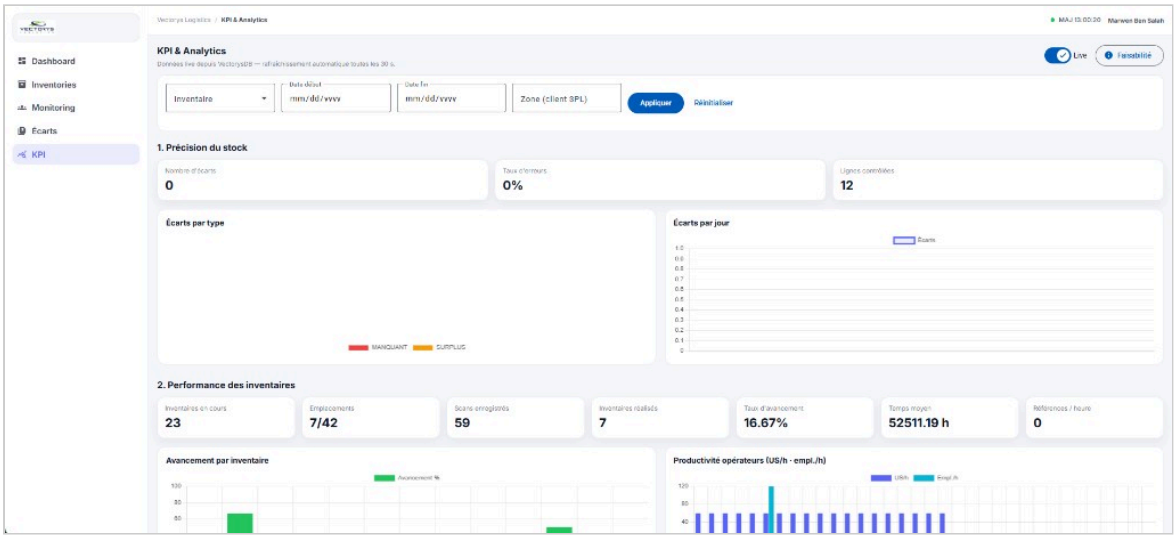


Figure 4.8 — Tableau de bord KPI réalisé

Le paramétrage CORS de l'API autorise notamment `http://localhost:4200` en développement. Sur le serveur local de démonstration, cette origine est remplacée par l'URL réelle du portail (adresse IP ou nom d'hôte du serveur), afin que les appels XHR et SignalR restent conformes à la politique du navigateur.

4.4 Réalisation de l'application mobile agent

L'application Flutter est structurée par fonctionnalités (`features/auth`, `scan`, `offline_sync`, `agent`) avec une couche réseau Dio, un stockage local SQLite et une configuration d'URL API adaptable (`API_BASE_URL` via `--dart-define`).

4.4.1 Authentification et navigation

L'agent s'authentifie par code personnel. Après succès, un shell de navigation donne accès au tableau de bord, à la liste des emplacements assignés, à la synchronisation et aux réglages. Seuls les emplacements de l'agent connecté, rattachés à un inventaire *EN_COURS*, sont exposés par `GET /api/emplacements/mes-emplacements`.



Figure 4.9 — Liste des emplacements assignés (mobile)

4.4.2 Boucle de scan et quotas

La page de boucle de scan orchestre :

1. la lecture du code-barres (caméra / scanner) ;
2. l'envoi de `POST /api/scans` (ou file locale si hors ligne) ;
3. l'affichage de la progression par code US (quantité scannée / quantité théorique) ;
4. le passage automatique de l'emplacement à *TERMINE* lorsque le quota global est atteint ;
5. le refus des scans supplémentaires (« quota atteint »).

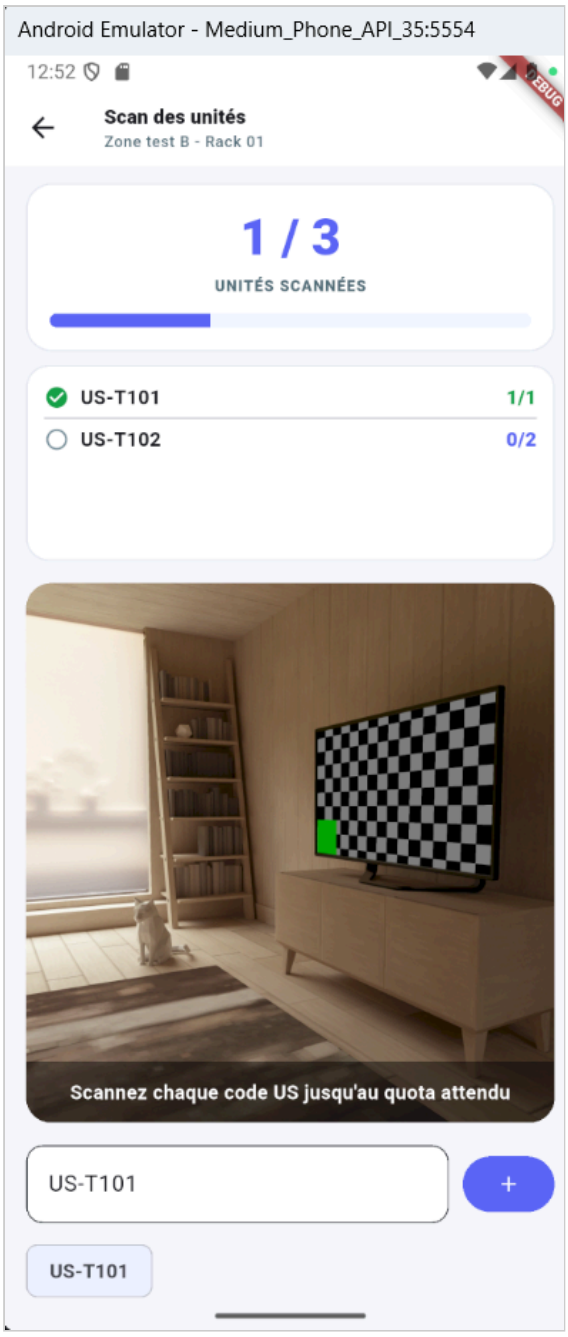


Figure 4.10 — Boucle de scan et progression par unité de stock

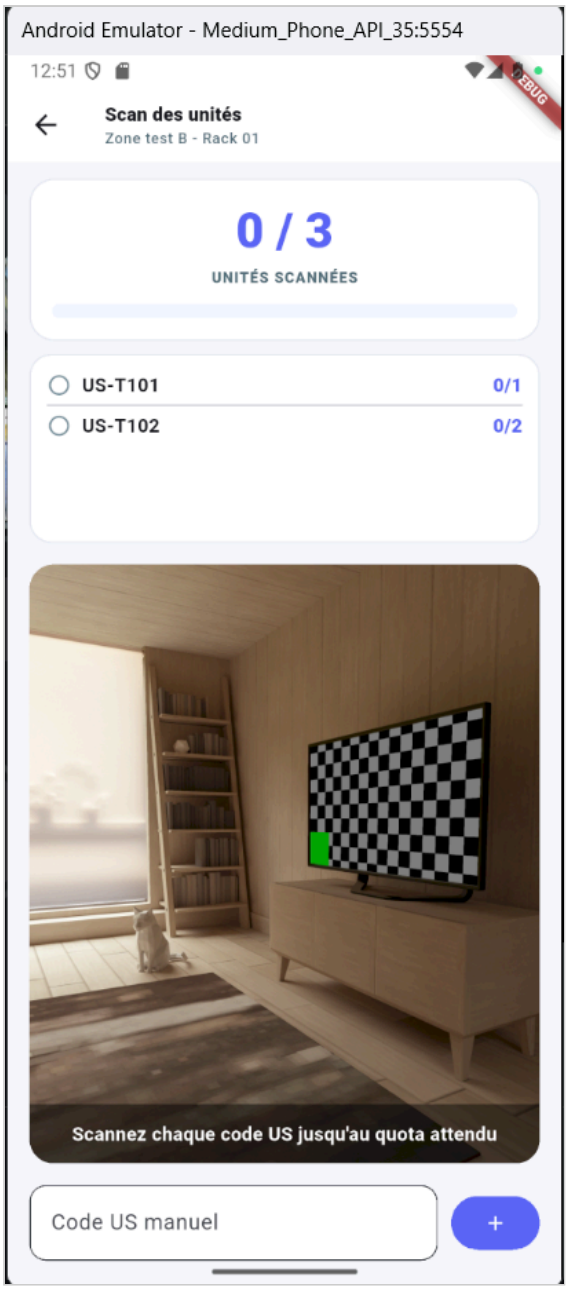


Figure 4.11 — Démarrage du comptage sur un emplacement

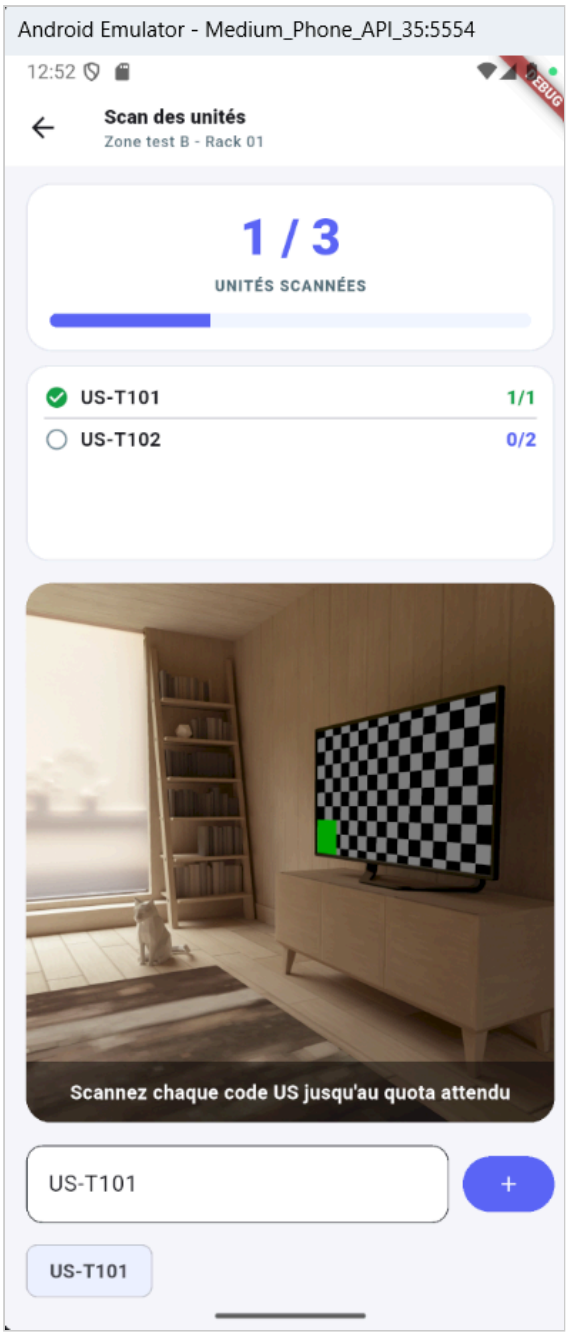


Figure 4.12 — Progression du scan pendant le passage C1

4.4.3 Mode hors ligne et synchronisation

Lorsque la connectivité chute, les repositories « offline-aware » enregistrent les scans (et, le cas échéant, les fins de comptage) dans SQLite avec un identifiant local. Un service `OfflineSyncService` observe le retour réseau, pousse un `POST /api/scans/batch`, gère les doublons et met à jour une bannière / page de synchronisation destinée à l'agent.

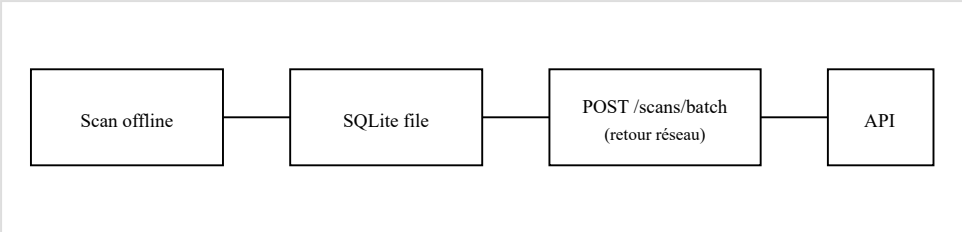


Figure 4.13 — Chaîne de synchronisation offline réalisée sur Flutter

4.5 Déroulement des sprints et livrables

Le tableau ci-dessous relie chaque sprint aux artefacts techniques principaux. Il complète la frise du chapitre 1 en précisant *ce qui a été réalisé* dans le code et les interfaces.

Sprint	Objectif	Livrables techniques
0	Fondations	Solution .NET Clean Architecture, projet Flutter, dépôt Git, SQL Server, script de setup DB
1	Authentification	Login JWT multi-rôle, policies, login Angular, login Flutter, seed utilisateurs
2	Inventaire & import	CreateInventaire + C1 auto, ImportStock ClosedXML, assignation CodeAgent, UI création / import
3	Scan & temps réel	EnregistrerScan, quotas, auto-TERMINE, InventaireHub, suivi-temps-reel, UI monitoring
4	Offline	SQLite, OfflineSyncService, POST /scans/batch, bannière et page de sync
5	Écarts (branche A)	DetecterEcarts, AppliquerBrancheA (C2 ciblé), UI liste d'écarts
6	Clôture & consolidation	Workflows de recomptage / finalisation inventaire, export résultats, extensions branche B selon besoin terrain
7	KPI	KpiController + repository agrégé, index SQL, dashboard Chart.js, filtres
8	Tests & doc	Swagger annoté Bearer, scénarios manuels Postman / guide d'import, consolidation des jeux de test
9	Mise en service locale	Publication API + portail sur serveur local, APK paramétré vers l'IP serveur, validation E2E



Figure 4.14 — Enchaînement des sprints de réalisation

4.6 Déploiement sur serveur local

Conformément au contexte entreprise, la mise en service retenue pour la démonstration et l'exploitation interne n'est pas un hébergement cloud public, mais un **déploiement sur serveur local** (machine Windows du réseau VECTORYS). Ce choix limite les contraintes de sécurité externe, rapproche l'API de SQL Server Express et autorise les terminaux mobiles à joindre l'API via l'adresse IP privée du serveur.

4.6.1 Architecture cible de déploiement

Composant	Déploiement local prévu
SQL Server	Instance locale ou machine serveur (`VectorysDB`), Trusted Connection ou compte SQL dédié
API ASP.NET Core	`dotnet publish` puis exécution Kestrel (service Windows) ou site IIS local
Portail Angular	`ng build` ; fichiers statiques servis (IIS / reverse proxy) vers le réseau LAN
Application mobile	APK généré avec `--dart-define=API_BASE_URL=http://<IP-serveur>:<port>/api`
CORS / SignalR	Origins = URL du portail ; ouverture du port API sur le pare-feu Windows

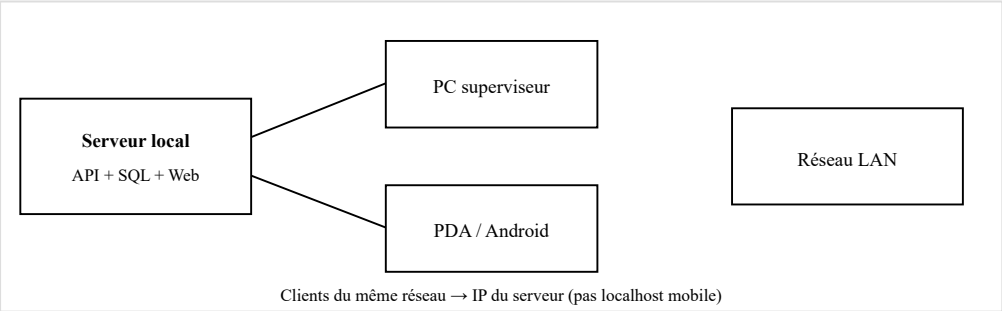


Figure 4.15 — Topologie de déploiement sur serveur local



Figure 4.16 — API publiée sur le serveur local (documentation Swagger)

[Insérer ici une capture : portail Angular accessible via l'IP / nom du serveur]

Figure 4.17 — Portail superviseur déployé en local

[Insérer ici une capture : APK / réglages pointant vers l'URL API du serveur]

Figure 4.18 — Application mobile configurée pour le serveur local

4.6.2 Points de configuration critiques

1. **Chaîne de connexion** — pointer vers l'instance SQL du serveur dans `appsettings.Production.json` (ou variables d'environnement) ;
2. **Secrets JWT** — remplacer la clé de développement par un secret fort hors dépôt ;
3. **URL Flutter** — ne plus utiliser `10.0.2.2` (émulateur) ni `localhost` depuis un téléphone physique ;
4. **HTTPS** — optionnel en LAN contrôlé ; recommandé si exposition plus large ;
5. **Pare-feu** — autoriser le port de l'API (ex. 5000) et du site web.

4.7 Difficultés rencontrées et solutions retenues

Difficulté	Impact	Solution retenue
Zones entrepôt sans couverture Wi-Fi stable	Perte de scans, reprise manuelle coûteuse	File SQLite + sync batch avec dédoublonnage côté API
Risque de clôture anticipée d'emplacement	Sous-comptage, écarts artificiels	Terminaison automatique pilotée par QteTheorique ; fin manuelle désactivée
Assignation ambiguë des emplacements	Agents travaillant hors périmètre	Colonne CodeAgent obligatoire à l'import ; filtrage strict mes-emplacements
Besoin de supervision instantanée	Retards de pilotage si simple polling	Hub SignalR par groupe inventaire + endpoint de suivi
URL API différente selon émulateur / device / serveur	Échecs de connexion silencieusement confus	Résolution conditionnelle + surcharge `API_BASE_URL`
Fichiers Excel hétérogènes issus de l'ERP	Imports partiels ou rejetés	Schéma de colonnes figé, validation ligne à ligne, rapport d'import explicite

Ces arbitrages confirment la priorité donnée à la *robustesse terrain* plutôt qu'à la richesse purement esthétique des interfaces. Ils seront réévalués lors des retours d'utilisation après déploiement local.

Conclusion

Le chapitre 4 a présenté la réalisation technique de Vectorys Inventaire : outillage, API Clean Architecture (CQRS, JWT, EF Core, SignalR, ClosedXML), portail Angular de supervision, application Flutter agent (scan, quotas, offline), enchaînement des sprints et stratégie de mise en service sur serveur local. Les figures illustrent les principaux écrans livrés ; les emplacements 4.16 à 4.18 permettront d'intégrer les captures de déploiement. Le chapitre 5 abordera les tests, la validation fonctionnelle et les perspectives d'évolution de la plateforme.